Labyrinth Breach: A Reproducible Unity ML-Agents Benchmark for Asymmetric Multi-Agent Pursuit-Evasion in Dynamic Mazes

Ahmed Jawed, Muhammad Sikander Raheem, and Usman Irshad Bhatti

***Abstract*: We present Labyrinth Breach, a configurable Unity ML-Agents benchmark for asymmetric multi-agent pursuit-evasion in dynamic maze environments. Three Sentinel agents cooperatively pursue two Runner agents across a seeded training maze and procedurally generated held-out topologies with shifting walls, exit zones, and partial observability. Agents observe through vector states, a 360-degree planar horizontal ray fan, and last-known-position memory, and are trained with PPO and asymmetric self-play under a tactical reward-shaping framework designed to reduce free-rider collapse, cluster convergence, and passive evasion. Beyond the environment itself, we contribute an auditable evaluation pipeline that routes raw Unity logs into run-scoped artifacts, computes outcome and coordination metrics, and supports multi-seed seen/unseen layout evaluation and ablation studies. Preliminary legacy runs suggest that curriculum training improves behavior as environments progress from open arenas to dynamic mazes, but publication-grade claims require the stricter multi-seed protocol described in this paper. The source code and trained models are publicly available.[1] A demonstration video of trained agents in the dynamic maze is available online.[2]**

***Index Terms*: Multi-agent reinforcement learning, pursuit-evasion, dynamic maze, asymmetric self-play, reward shaping, Unity ML-Agents, PPO**

I. Introduction

Pursuit-evasion is a classical problem in robotics and multi-agent decision-making. One side attempts to capture while the other attempts to escape. The problem is a natural fit for reinforcement learning (RL) because the desired behavior is difficult to hand-code when the environment involves adversarial interaction, obstacles, partial information, and multiple cooperating agents.

The original course project requires RL-based multi-agent pursuit-evasion using Unity ML-Agents [1]. We extend this requirement into a structured environment called *Labyrinth Breach*, where three Sentinels pursue two Runners inside a maze with exit zones and dynamic topology changes.

Simple chase environments do not capture the full difficulty of multi-agent coordination. In a richer maze setting, effective Sentinel behavior includes trapping, pincer movement, corridor denial, exit blocking, and route cutting. On the other side, effective Runner behavior includes distance management, path selection, survival, and escape through exits. These requirements align the project with robotics-style navigation and coordination problems rather than only direct-motion pursuit.

Our contributions are as follows:

- A configurable asymmetric 3v2 pursuit-evasion environment with a fixed training topology, seeded placements, dynamic walls, and procedurally generated held-out mazes.
- A tactical reward shaping framework (`v5_active_agents`) that targets free-rider collapse, clustering, and passive evasion through configurable shaping terms.
- A memory-augmented partial observation design combining a 360-degree planar horizontal ray fan with last-known-position tracking.
- A four-stage curriculum pipeline (fixed static $\rightarrow$ randomized static $\rightarrow$ low-frequency dynamic $\rightarrow$ high-frequency dynamic) with transfer between stages.
- A reproducible evaluation and evidence pipeline for multi-seed seen/unseen testing, coordination metrics, and ablations over memory, tactical rewards, dynamic walls, curriculum, and action assistance.

The study is organized around three research questions. **RQ1** asks whether a staged curriculum can produce competent policies as navigation complexity increases from an open arena to a dynamically changing maze. **RQ2** asks how strongly a fixed policy depends on maze geometry when transferred to held-out procedural layouts. **RQ3** asks which design elements, including memory, tactical reward shaping, wall dynamics, curriculum transfer, and action assistance, are responsible for observed performance and coordination. The current artifact answers only the pipeline-validity and fixed-checkpoint part of RQ2. RQ1 and RQ3 require the independent-seed and ablation experiments specified later in this article.

This distinction between a *benchmark contribution*, a *validated diagnostic*, and a *completed empirical study* is important. The environment and evidence pipeline are implemented and auditable. The diagnostic establishes that a legacy policy can execute under the corrected protocol. The final comparative claims remain hypotheses until the official experiment matrix is complete. We therefore report positive and negative evidence, including a layout where the fixed policy performs poorly, and avoid treating successful execution as proof of robust generalization.

The authors are with the MS Artificial Intelligence program at Lahore University of Management Sciences (LUMS), Lahore, Pakistan. This work originated in AI641: AI for Robotics.

[1] https://github.com/ahmedjawedaj/labyrinth-breach-marl

[2] https://drive.google.com/file/d/1sxzKtpvVNh_UYYxchMTZzzZNKLDyh09w/view

II. Related Work

We organize the related work into five themes and compare the closest systems in Table I. The comparison exposes the intended niche of Labyrinth Breach: asymmetric teams, continuous control, partial observability, goal-directed evasion, dynamic topology, and run-level provenance in one configurable Unity benchmark. Table II then positions the current empirical evidence against recent journal systems. The comparison is deliberately conservative: published pursuit-evasion environments differ in dynamics, sensing, team size, terminal definitions, and evaluation budgets, so cross-paper numbers are evidence of scope and difficulty rather than a controlled leaderboard. The comparison also makes clear what the present system does not yet provide, most importantly physical-robot validation.

A. Pursuit-Evasion as a Multi-Agent Problem

Direct pursuit-evasion papers share the same capture-versus-escape structure as our task. In harder settings, useful behaviors go beyond direct motion and include route blocking, coordinated pressure, spatial entrapment, and strategic escape timing [5], [8], [13], [16], [20], [27]. De Souza Jr. et al. train decentralized pursuit policies for non-holonomic agents and transfer them to three physical drones [28]. Gonultas and Isler incorporate vehicle dynamics and sensor constraints and deploy learned policies on ground robots [8]. ViPER uses graph attention for visibility-based search and includes an aerial hardware demonstration [13]. These systems set a stronger robotics validation bar than simulation alone.

Labyrinth Breach studies a complementary configuration: three cooperative pursuers against two learned, goal-directed evaders in a maze whose topology can change during an episode. Capture is not the sole objective because Runners can actively reach exits, and the 3v2 asymmetry creates both within-team coordination and between-team adaptation. The contribution is therefore not a claim to replace dynamics-faithful or hardware-validated pursuit systems. It is a configurable testbed for studying how environment structure, sensing, memory, and reward design interact under reproducible MARL evaluation.

Recent journal work also shows how pursuit-evasion has moved beyond simple open-field capture. Du et al. model unauthorized-UAV pursuit in urban airspace with MARL [21]. Peng et al. add limited visual fields, building occlusion, graph attention, and normalizing-flow policies for multi-UAV capture [22]. Qu et al. use centralized training, decentralized execution, LSTM memory, multi-agent soft actor-critic, and multi-stage reward guidance for cooperative USV pursuit [23]. Xu and Dang analyze emergent pursuit behaviors in a bounded grid world and report a 99.9% pursuer success rate over 1,000 randomized trials, but under a simpler discrete, complete-information setting [24]. These papers strengthen the need to report environment assumptions alongside performance.

B. Cooperative and Mixed MARL

Multi-agent reinforcement learning faces a non-stationarity challenge because all agents update simultaneously. MADDPG addressed this through centralized training with decentralized execution [2]. Yu et al. subsequently showed that PPO-based methods can be strong cooperative baselines when implementation details and hyperparameters are controlled carefully [4]. Our two policy groups use parameter sharing within roles: all Sentinels share one policy, and all Runners share another. This exploits role homogeneity without forcing the two sides to share observations, objectives, or network weights.

Unity ML-Agents PPO is deliberately retained as the primary baseline because it is integrated with the simulator and straightforward to reproduce. This choice should not be interpreted as an algorithmic claim. MAPPO offers a centralized critic, and MA-POCA is designed for cooperative groups whose membership or participation may change. A journal-grade algorithm comparison should hold environment steps, network capacity, evaluation budgets, and seed counts constant. The present extended manuscript specifies that comparison as future empirical work rather than implying superiority from one PPO training trace.

C. Credit Assignment

When multiple agents cooperate under team-level objectives, one agent may contribute by blocking a route or creating pressure while another gets the final capture. Plain shared reward can be too coarse for learning such indirect contributions [3], [6], [7], [17]. Recent methods decouple teammate reward streams [31] or reason over individual, joint, and correlated agent subsets [33]. A 2026 model-based pursuit-evasion system further combines adversary uncertainty with mutual policy advising during imagined rollouts [34]. Labyrinth Breach takes a simpler environment-level approach: event detectors identify pincer pressure, corridor control, exit denial, enclosure, and capture, after which role-specific reward policies assign configured bonuses.

This mechanism makes the source of each shaping reward observable in `reward_audit.csv`, but it does not solve credit assignment in the formal counterfactual sense. A detector may reward correlated behavior that was not causally responsible for a capture. For that reason, the final ablation protocol removes the tactical layer entirely and reports both outcome effects and event-frequency changes. If the coordination metrics do not degrade when the layer is removed, the claimed mechanism is unsupported even if total return changes.

D. Partial Observability

Pursuit-evasion becomes more meaningful when sensing is limited. With full global state, the task is easier and less realistic. Works on visibility-constrained pursuit-evasion show that line-of-sight, uncertainty, and memory fundamentally change the problem [8], [9], [13]. R2PS goes further by maintaining a belief over possible evader nodes and evaluating two pursuers against an optimal asynchronous evader on 10 unseen real-world graphs [15]. Its 300-graph training set and 500 tests per graph establish a stronger partial-observability standard than a last-known-position feature. Our observation pipeline

uses vector states combined with ray-based visibility and last-known-position memory rather than global state or camera images. The memory is intentionally low-capacity: it stores the last observed relative position of a target and a recency signal. It cannot infer a hidden trajectory or represent multimodal beliefs, so recurrent or explicit-belief memory remains an important baseline.

E. Curricula, Benchmarks, and Evaluation Practice

Curriculum learning is especially relevant when sparse terminal rewards and adversarial co-adaptation make direct training unstable. Grupen et al. combine environment and behavioral curricula in pursuit-evasion and analyze emergent implicit signaling [29]. DualCL combines an intrinsic-parameter curriculum with an environment generator and evaluates three seeds with 3,000 episodes per reported score, including two out-of-distribution scenes [11]. MatrixWorld connects pursuit-evasion, safety-constrained action execution, co-evolution, and autocurricula in a lightweight grid benchmark [9]. Labyrinth Breach instead uses a manually specified progression that preserves network dimensions across stages, enabling checkpoint transfer while introducing walls and then topology shifts. The direct-dynamic ablation is therefore necessary to separate curriculum benefit from training budget.

Cross-topology claims require more than changing placement seeds. Equilibrium Policy Generalization trains graph policies over 152 graphs and evaluates 10 real-world plus 10 held-out Dungeon graphs with 500 trials per graph [14]. It also uses an equilibrium oracle, so its game-theoretic interpretation does not transfer to PPO self-play here. We consequently use the narrower term "held-out topology transfer" and report each topology separately. Zheng et al. show a complementary mechanism: obstacle-aware rewards can improve encirclement, but their five-seed, 200,000-episode study evaluates success, efficiency, team size, obstacle count, and component ablations together [12]. This motivates testing whether our trap reward changes terminal outcomes and capture efficiency rather than merely increasing shaped return.

Evaluation methodology is a separate concern from environment design. Gorsane et al. identify inconsistent seed counts, evaluation budgets, and uncertainty reporting across cooperative MARL studies and propose a standardized protocol [30]. Our official protocol follows the same principles at smaller project scale: independent training seeds, fixed evaluation manifests, held-out tasks, per-seed reporting, confidence intervals, and preservation of raw artifacts. The validated diagnostic uses only one checkpoint pair and therefore cannot substitute for this protocol.

F. Hybrid and Assisted Control

Hybrid controllers combine learned actions with a geometric, model-based, or heuristic control signal. Akinmolayan et al. blend a classical interception vector with PPO and anneal the blend during training [32]. Labyrinth Breach contains a related optional controller feature: the Sentinel action can be blended with a direction toward the tracked Runner. The active diagnostic configuration uses a Sentinel pursuit strength of 0.15. This makes action assistance a potential confound, not an implementation detail. The coefficient must be disclosed and assist-off performance must be reported.

G. Explainability and Policy Interpretation

Explainable reinforcement learning is increasingly relevant for autonomous robots because strong performance alone does not show why a learned policy acts as it does. Cetin et al. apply SHAP to deep RL counter-drone decisions and show how feature attribution can make an interception policy more transparent [25]. A recent XMARL survey frames interpretability at multiple levels: individual decisions, interaction mechanisms, team strategy emergence, and system-level performance [26]. Labyrinth Breach follows this multi-level framing with artifact-level explanations rather than image saliency. Raw observations and actions support individual inspection, replay events expose interaction mechanisms, coordination metrics summarize emergent team strategies, and paired ablations test system-level dependencies. Full SHAP analysis is left as an extension because our deployed PPO policies use role-specific continuous actions, ray groups, memory features, and multi-agent coupling. Applying SHAP without careful feature grouping would risk producing visually attractive but weak explanations.

III. Problem Formulation

We model the environment as a Partially Observable Stochastic Game (POSG) defined by the tuple

$$\mathcal{G} = \langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, P, \{\Omega_i\}, O, \{R_i\}, \gamma \rangle, \tag{1}$$

where $\mathcal{I} = \mathcal{I}_S \cup \mathcal{I}_R$ contains three Sentinels and two Runners. The global state $s_t \in \mathcal{S}$ contains agent poses and velocities, capture/escape states, exits, procedural maze state, and the active wall set. Each action $a_{i,t} = [u^x_{i,t}, u^z_{i,t}] \in [-1, 1]^2$ controls planar movement. Agent i receives only $o_{i,t} = O_i(s_t) \in \Omega_i$, with 117 dimensions for a Sentinel and 129 for a Runner. The Unity transition kernel $P(s_{t+1} \mid s_t, \mathbf{a}_t)$ combines deterministic control code, collision handling, and seeded procedural events.

Agents are homogeneous within a role but not across roles. Parameter sharing therefore gives

$$\pi_{\theta_S}(a_{i,t} \mid o_{i,t}),\ i \in \mathcal{I}_S, \qquad \pi_{\theta_R}(a_{j,t} \mid o_{j,t}),\ j \in \mathcal{I}_R. \tag{2}$$

For role $q \in \{S, R\}$, PPO seeks parameters maximizing the expected discounted return against an opponent-policy mixture μ_{-q},

$$J_q(\theta_q, \mu_{-q}) = \mathbb{E}_{\pi_{\theta_q},\, \pi_{-q} \sim \mu_{-q},\, P} \left[\sum_{t=0}^{T} \gamma^t r_{q,t} \right]. \tag{3}$$

Our self-play controller uses $\mu_{-q} = 0.5\delta_{\theta^{\text{latest}}_{-q}} + 0.5\,\text{Unif}(\mathcal{H}_{-q})$, where $\mathcal{H}_{-q}$ is a window of the ten most recent opponent snapshots. One role learns at a time. The learning role changes every 100,000 behavior steps, snapshots are saved every 20,000 steps, and the deployed opponent is resampled every 2,000 steps. This limits direct latest-policy cycling but does not make the game stationary. The environment is mixed-sum rather than strictly zero-sum. Terminal rewards

TABLE I

COMPARISON WITH RECENT LEARNING-BASED PURSUIT-EVASION SYSTEMS AND BENCHMARKS. "DYNAMIC TOPOLOGY" MEANS OBSTACLES CHANGE CONNECTIVITY DURING AN EPISODE, NOT ONLY THAT AGENTS MOVE.

Work	Teams	Control	Sensing	Environment	Validation	Primary emphasis
De Souza Jr. et al. [28]	Multi-pursuer, one evader	Non-holonomic	Local ordered neighbors	Open pursuit area	Simulation + drones	Decentralized pursuit and formation
Kouzeghar et al. [10]	Pursuer + scout roles	Aerial	Local neighbors	Random obstacles	Simulation + drones	Pursuit and scouting
DualCL [11]	Multi-pursuer, one evader	3-D aerial	Global state	Generated obstacles	Simulation + drones	Dual curriculum and transfer
Gonultas and Isler [8]	1v1	Car-like dynamics	Sensor constrained	Indoor obstacles	Simulation + ground robots	Dynamics and sensing constraints
MatrixWorld [9]	Variable	Discrete grid	Configurable	Static grid topology	Simulation	Safety, co-evolution, autocurricula
ViPER [13]	Multi-pursuer, evaders	Planar/aerial	Visibility limited	Multiply connected maps	Simulation + aerial hardware	Distributed visibility clearing
EPG/R2PS [14], [15]	Graph pursuit	Discrete graph	Full/partial	Cross-graph	Simulation	Topology transfer and robust belief
Akinmolayan et al. [32]	4v3 extension	Hybrid continuous	Radar + communication	Bounded defense arena	Simulation	Classical/PPO blending and self-play
Labyrinth Breach	3v2 learned teams	Continuous planar	Rays + vector memory	Changing 3-D maze	Unity simulation	Topology shifts, exit evasion, auditable metrics

TABLE II

STATE-OF-THE-ART POSITIONING. REPORTED PERFORMANCE IS TAKEN FROM THE ORIGINAL PAPERS WHEN A DIRECTLY STATED NUMBER IS AVAILABLE. DIFFERENCES IN ENVIRONMENT, SENSING, DYNAMICS, AND EPISODE RULES PREVENT DIRECT RANKING.

Work	Setting	Reported evidence in source work	Position relative to Labyrinth Breach
Du et al. [21]	Urban unauthorized-UAV pursuit with MARL	Journal study of cooperative UAV pursuit in urban airspace	Stronger urban-airspace framing. Labyrinth adds learned 3v2 evaders, exit objectives, changing maze topology, and run-provenance audit
De Souza Jr. et al. [28]	Decentralized multi-drone pursuit	Simulation plus physical drone transfer for non-holonomic agents	Stronger sim-to-real evidence. Labyrinth studies asymmetric two-team evasion and dynamic maze connectivity
Peng et al. [22]	Multi-UAV pursuit with limited visual field and building occlusion	NAGC compared with MARL baselines in high-precision simulation, including ablations and generalization scenarios	Stronger algorithmic comparison. Labyrinth contributes a reproducible Unity benchmark with exit-directed evasion and artifact-gated evaluation
Qu et al. [23]	Cooperative USV pursuit using MASAC, CTDE, LSTM, and staged reward guidance	Simulation evidence for cooperative capture against a dynamic target	Stronger maritime-domain specificity. Labyrinth contributes multi-role adversarial learning and dynamic indoor-like topology
Xu and Dang [24]	Bounded 2-D grid pursuit-evasion with MARL behavior mining	99.9% pursuer success in 1,000 randomized trials plus clustered behavior analysis	Stronger quantitative behavior mining in a simpler complete-information grid. Labyrinth uses continuous control, partial observability, and two goal-directed evaders
Labyrinth Breach	Unity 3v2 dynamic maze, assist-off canonical evaluation	30 policy-layout cells and 3,002 episodes: 42.2% Sentinel win rate, 53.5% escape rate, 20.3% held-out pincer episode rate	Not a SOTA-performance claim. Evidence supports a difficult, balanced, auditable benchmark and motivates remaining retraining ablations

are opposing, but role-specific shaping rewards need not sum to zero. Consequently, neither a Nash-equilibrium claim nor convergence to a game-theoretic solution follows from stable training curves or ELO values.

The Sentinel team wins when both Runners are captured before timeout. The Runner side wins when at least one active Runner reaches an exit zone or when the episode reaches the active rule-config timeout. Official dynamic-maze runs use 180 seconds. Excluded legacy plots may use the earlier 120-second regime. Agent components use environment-managed termination rather than a serialized per-agent `MaxStep`. This detail matters because a trainer step is not equivalent to an episode step when five agents share two policies.

Let $c_j(t) \in \{0,1\}$ indicate that Runner j is captured and $e_j(t) \in \{0,1\}$ indicate escape. The terminal outcome is

$$y = \begin{cases} S, & \prod_{j \in \mathcal{I}_R} c_j(T) = 1, \\ R, & \max_j e_j(T) = 1 \text{ or } T = T_{\max}. \end{cases} \tag{4}$$

The episode-level Sentinel win estimator over n completed episodes is $\hat{p}_S = n^{-1} \sum_{k=1}^{n} \mathbb{1}[y_k = S]$. We use Wilson score intervals for this binary proportion because the diagnostic contains small cells and rates near zero or one, where a normal approximation is unreliable.

Dynamic topology is represented by a time-indexed traversability graph $G_t = (V, E_t)$. At a wall event, a seeded controller changes a bounded subset ΔE_t subject to a safety buffer around agents. Exit reachability follows from the generator invariant that spanning-tree passages are never converted into dynamic edges. It does not rely on the currently inert `ExitGuard` tag check. Thus $E_{t+1} = E_t \triangle \Delta E_t$. The policy does not observe G_t globally. It must detect local changes through rays and context features. This produces a different problem from static procedural randomization, where a layout changes only between episodes.

This formulation supports the study of asymmetric team interaction, coordinated pursuit, tactical escape behavior, partial observability, dynamic environment changes, and generalization to unseen layouts.

IV. System Design

Figure 1 summarizes the complete workflow. The core design choice is that simulation, learning, evaluation, and reporting are not treated as separate manual steps. Instead, every paper result is generated from run-scoped artifacts and auditable metadata. This is intended to reduce the common MARL failure mode in which training curves, screenshots, and hand-selected demonstrations cannot be traced back to reproducible seeds and configurations.

A. Environment Structure

The implementation includes four scenes of increasing complexity:

- **Open arena baseline:** flat arena with no walls.
- **Static maze:** a fixed corridor topology with configurable placements.
- **Dynamic maze:** walls shift at configurable intervals during episodes.
- **Unseen evaluation maze:** novel layouts not encountered during training.

The dynamic maze is the core environment. Training uses a fixed base topology, while held-out evaluation topologies are generated deterministically from reserved seeds. In both cases, selected wall pillars can rise or lower on a timer. These shifts invalidate a fixed route during an episode and create trapping opportunities and escape adaptations in real time. Exit zones are placed explicitly for the Runner side, making evasion a goal-directed navigation task rather than pure survival.

Figure 2 provides the environment visual requested by reviewers. It emphasizes the state variables and behavioral events used by the evaluation pipeline rather than presenting a decorative screenshot. This makes the maze, agent roles, dynamic-wall mechanism, and qualitative behavior categories clear before the training details are introduced.

B. Seeded Topology Generation

Held-out mazes use a 13×13 occupancy grid containing a 6×6 lattice of candidate free cells. A seeded randomized depth-first search carves a spanning tree E_{tree} over all 36 cells. Four additional non-tree connections are opened permanently to reduce single-corridor degeneracy, and up to five further non-tree connections are marked as dynamic wall edges. The traversable graph is therefore

$$E_t = E_{\text{tree}} \cup E_{\text{loop}} \cup E_{\text{dyn}}(t), \qquad E_{\text{dyn}}(t) \subseteq E_{\text{optional}}. \quad (5)$$

Because E_{tree} is never removed, every generated topology remains connected even when all dynamic connections are raised. This is a construction-level safety property, not an empirical assumption. The reserved topology seeds are 101, 202, 303, 404, and 505. Topology seed q_l is fixed within policy-layout cell l, whereas placement seed $q_l + k$ varies with episode k. Each episode row records both `maze_layout_id` and `maze_topology_seed`. This prevents placement randomization from being mislabeled as topology diversity.

C. Agent Architecture

The codebase separates agent logic, environment control, reward computation, sensing, and logging into distinct modules. `BaseAgent` handles movement, observation collection, and ML-Agents integration. `SentinelAgent` extends it with capture-radius logic and target tracking. `RunnerAgent` extends it with escape and captured-state management. The central `PursuitEvasionEnvController` manages episode lifecycle, capture detection, reward triggering, and terminal conditions.

Figure 3 makes the model boundary explicit. The neural policy consumes grouped vector observations and outputs a two-dimensional continuous movement command, while Unity handles collision, capture, escape, optional assistance, and artifact logging outside the network. This separation is important because the optional geometric assistance and the evaluation-only baselines are controller-level interventions, not additional learned layers.

The controller can optionally blend a learned direction $a_{i,t}$ with a geometric direction $h_{i,t}$:

$$\widetilde{a}_{i,t} = (1 - \alpha_{i,t}) a_{i,t} + \alpha_{i,t} h_{i,t}, \quad (6)$$

$$\alpha_{i,t} = \text{clip}(\rho_q [c_q + (1 - c_q) u_{i,t}], 0, 1). \quad (7)$$

where q is the role, ρ_q is the configured assist strength, and $u_{i,t}$ increases as the visible or remembered target becomes nearer. The source uses $c_S = 0.35$ for pursuit and $c_R = 0.30$ for evasion, with obstacle-aware sliding applied to $h_{i,t}$. Training uses $\rho_S = 0.15$ and $\rho_R = 0$, but the canonical publication evaluation sets both to zero so reported policy behavior is unassisted. Since $h_{i,t}$ is computed outside the policy network, the matched assist-on intervention reinstates $\rho_S = 0.15$ and measures a hybrid controller while retaining the checkpoint, topology, placements, and every other rule parameter.

D. Observation Design

The observation pipeline is state-based rather than camera-based. Table III shows the complete breakdown. Each agent receives a flat vector containing self-state (position, velocity, heading, alive flag), environment context (time remaining, nearest exit distance and direction, wall proximity, wall shift timer), a fixed-height 360-degree planar horizontal ray fan,

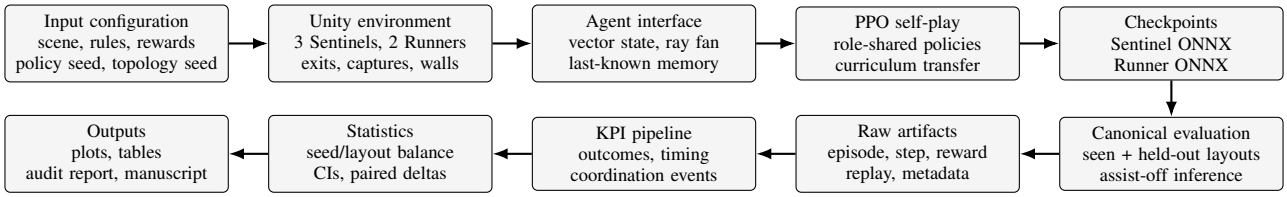

Fig. 1. Proposed methodology and evidence flow. The same seeded configuration controls training, evaluation, logging, KPI extraction, statistical analysis, and manuscript figures. Strict artifact validation excludes runs with missing logs, unknown seeds, or mismatched scene/rule/reward provenance from publication tables.

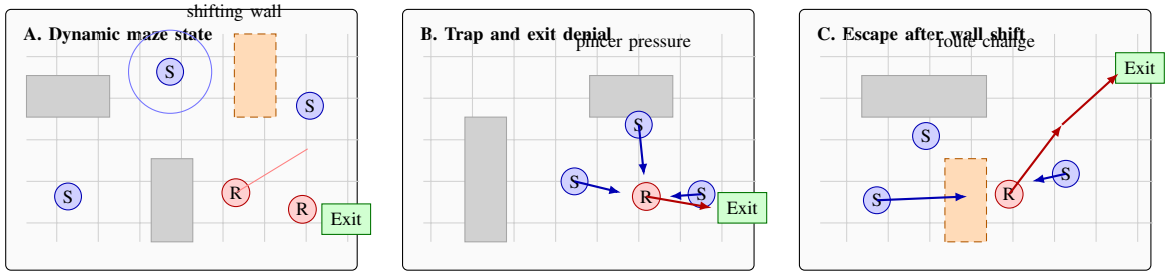

Fig. 2. Environment and qualitative behavior schematic. The panels show the core visual elements that matter for the method: dynamic walls, exit zones, local sensing, pincer pressure, exit denial, and route change after topology shifts. The figure is schematic rather than a raw Unity screenshot so the experimental variables remain legible in print.

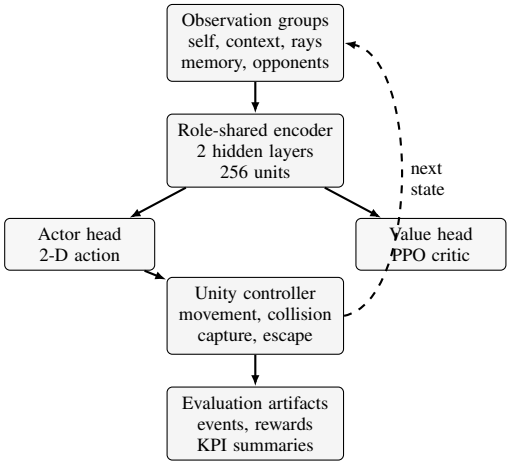

Fig. 3. Role-specific PPO architecture used by the Sentinel and Runner policies. The figure describes the implemented ML-Agents actor-critic network and runtime controller path. It is not a transformer or attention architecture.

last-known-position memory for targets that have left line of sight, and a summary of the two nearest opponents including their relative position, visibility, and threat pressure.

Sentinels cast 14 rays while Runners cast 16 rays. Each ray produces 6 floats: normalized hit distance and one-hot indicators for wall, exit, teammate, opponent, and other object types. This gives Runners wider sensing coverage because their survival depends on detecting threats from all directions.

E. Reward Design

The reward system uses the `v5_active_agents` configuration, which was iteratively developed to address three behavioral failure modes observed during training: (1) free-rider collapse, where one Sentinel learns to capture while others idle, (2) cluster convergence, where all Sentinels chase the same Runner in a single file, and (3) passive evasion, where Runners survive by standing still rather than actively seeking exits.

TABLE III
OBSERVATION AND ACTION SPACE PER AGENT ROLE.

Component	Sentinel	Runner
Self state	10	10
Environment context	7	7
Ray perception (6 per ray)	84 (14 rays)	96 (16 rays)
Last-known-position memory	6	6
Opponent summary (2 nearest)	10	10
Total observation size	**117**	**129**
Action space	2 continuous	2 continuous

The reward function combines terminal win/loss signals with individual distance-based shaping for pursuit and evasion [19], anti-clustering penalties, idle penalties, exit-seeking shaping, and a tactical coordination layer that rewards events detected by a dedicated `TrapEventDetector` module. The tactical layer rewards pincer formations, corridor control, exit denial, and dead-end forcing. All weights are loaded from YAML configuration files at runtime. Table IV lists the final reward weights.

For agent i, the implemented per-step reward can be written as

$$r_{i,t} = r_{i,t}^{\text{terminal}} + \sum_{k \in \mathcal{D}} w_{i,k}\, g_k(\Delta d_{k,t}) + \sum_{m \in \mathcal{E}} w_{i,m}\, \phi_m(s_t, \mathbf{a}_t), \tag{8}$$

where $\mathcal{D}$ contains distance-progress terms and $\mathcal{E}$ contains discrete behavioral and tactical events. For a target distance

TABLE IV
REWARD WEIGHTS (v5_ACTIVE_AGENTS CONFIGURATION).

Sentinel Rewards	Weight
Team full capture (win)	$+1.0$
Timeout or escape (loss)	-1.0
Individual capture bonus	$+0.25$
Chase progress (per step)	$+0.004$
Chase regression	-0.003
Cluster penalty	-0.015
Idle penalty	-0.002
Wall loop penalty	-0.002
Exploration visit bonus	$+0.001$
Orbit stall penalty	-0.003
Runner Rewards	Weight
Team exit success (win)	$+1.0$
Both captured (loss)	-1.0
Tagged individually	-0.2
Survival step bonus	$+0.0001$
Exit approach bonus	$+0.005$
Exit approach regression	-0.002
Evade progress	$+0.001$
Threat approach penalty	-0.002
Wall loop penalty	-0.001
Exploration visit bonus	$+0.001$
Orbit stall penalty	-0.003

d_t, the shaping gate is

$$g(\Delta d_t) = \mathbb{1}(|\Delta d_t| > \delta) \min\left(1, \frac{|\Delta d_t|}{\kappa}\right), \tag{9}$$

with the sign and coefficient selected by whether the change is desirable. Sentinel chase progress uses $\Delta d_t = d_{t-1} - d_t$. Runner evasion uses $\Delta d_t = d_t - d_{t-1}$ while inside a configured threat radius. Exit approach again uses $d_{t-1} - d_t$. The source defaults use a 0.08 movement dead-zone for pursuit/evasion and 0.05 for exit approach.

The shaping terms are not proven potential-based and may therefore change the optimal policy, not merely accelerate learning. This is a deliberate engineering tradeoff, but it makes reward ablation essential. We evaluate outcomes separately from shaped return so that an increase in accumulated shaping reward cannot be mistaken for improved capture or escape behavior.

F. Logging and Reproducibility

Each training and evaluation run produces four CSV artifacts under a run-specific directory: `episode_log.csv` (per-episode outcomes and totals), `agent_step_log.csv` (per-step positions, actions, and distances), `reward_audit.csv` (per-event reward attribution by agent, category, and magnitude), and `replay_events.csv` (timestamped capture, escape, wall-shift, and tactical events). This structure ties every reported metric back to raw per-step evidence.

The Python launcher writes a second provenance layer containing the run manifest, seed, exact training or inference command, Git state, configuration snapshots, and SHA-256 hashes. Unity receives the repository root and requested scene through explicit environment variables. Runtime override files select curriculum, rule, and reward configurations. Episode rows repeat the effective scene and configuration names, allowing the audit to compare requested and observed provenance rather than trusting launcher metadata alone.

TABLE V
REVIEWER-FACING ARTIFACT MAP IN THE PUBLIC REPOSITORY.

Artifact	Purpose
Unity scripts	Agent control, sensing, reward logic, episode lifecycle, and CSV logging implementation
Configuration files	Trainer, reward, rule, curriculum, ablation, and evaluation manifests
Setup validator	Environment and repository preflight checks before experiments
Policy evaluator	Fixed-checkpoint inference launcher with metadata and log routing
Readiness auditor	Gate-level audit for manuscript, training, evaluation, ablation, and reproducibility status
Unit tests	Evaluation isolation and publication metric calculations
Evaluation summaries	Canonical assist-off 30-cell evaluation summaries and statistics
Diagnostic baselines	Random and geometric-heuristic action-override evaluations with aggregate CSV/JSON outputs
Official summaries	Learning curves, paired ablations, readiness reports, and power analysis outputs

A run is publication-valid only when all required artifacts are non-empty, the seed and run identifier are known, at least one episode is complete, and the observed scene/rule/reward tuple matches the manifest. This strictness detected an earlier 18-cell matrix in which the standalone silently loaded its default scene and configuration. That matrix is retained with an `INVALIDATED.md` marker but excluded from every reported result. The failure case demonstrates why artifact existence without semantic provenance is insufficient.

The same rule applies to retries. ML-Agents `-force` replaces trainer outputs but does not truncate CSV files written independently by Unity. A corrected training retry was therefore stopped when its V5 rows were found appended to legacy reward rows. The launcher now deletes the complete target run directory before a forced launch and writes an explicit `running` status before starting Unity. The mixed directory is preserved as invalidated audit evidence, never as a source of checkpoints or results.

V. TRAINING AND EVALUATION SETUP

A. Training Algorithm

We train both behaviors with PPO using parameter sharing within each team and asymmetric self-play across teams. All three Sentinels share a single policy network, and both Runners share a separate one. Table VI lists the official maze-stage hyperparameters. The excluded legacy open-arena trainer used a 1024 batch and a 64-step horizon and is not part of the four-stage official matrix. The learning rate follows a linear decay schedule over the training run. A draft MA-POCA YAML exists, but it is not a valid matched baseline: team-group registration, equalized budgets, self-play controls, and independent validation remain to be implemented. No MA-POCA result or support claim is made here.

TABLE VI
PPO HYPERPARAMETERS (SHARED BY BOTH BEHAVIORS).

Parameter	Static Maze	Dynamic Maze
Learning rate	3.0×10^{-4}	2.5×10^{-4}
Batch size	2048	2048
Buffer size	20480	40960
Entropy (β)	5.0×10^{-3}	4.0×10^{-3}
Clip range (ϵ)	0.2	0.2
GAE λ	0.95	0.95
Discount (γ)	0.99	0.99
Epochs per update	3	4
Hidden units	256	256
Hidden layers	2	2
Time horizon	128	160
Sentinel max steps	1,500,000	1,500,000
Runner max steps	1,500,000	1,500,000

For role q, let $r_t(\theta_q) = \pi_{\theta_q}(a_t \mid o_t)/\pi_{\theta_q^{\text{old}}}(a_t \mid o_t)$ and let $\widehat{A}_{q,t}$ be the generalized advantage estimate. The policy term optimized by PPO is

$$L_q^{\text{clip}}(\theta_q) = \mathbb{E}_t\big[\min\big(r_t(\theta_q)\widehat{A}_{q,t}, \text{clip}(r_t(\theta_q), 1-\epsilon, 1+\epsilon)\widehat{A}_{q,t}\big)\big]. \quad (10)$$

The implementation combines this term with a value loss and entropy bonus. This objective is applied independently to the Sentinel and Runner parameter sets. It does not centralize the critic across opposing roles. Checkpoint transfer initializes both θ_S and θ_R at the next curriculum stage, while optimizer progress and the new stage budget are recorded in run metadata.

B. Curriculum and Transfer Learning

Training follows a four-stage curriculum with transfer learning, motivated by evidence that staged difficulty helps agents discover coordination skills that are hard to learn from sparse reward alone [18], [29]. Each stage uses `-initialize-from` to bootstrap both role policies from the previous stage's checkpoint. The network architecture (256 hidden units, 2 layers) remains identical across stages so the parameter tensors are compatible.

Table VII reports the implemented progression from the canonical curriculum YAML. Stage 1 uses a fixed static maze and fixed spawn/exit locations. Stage 2 randomizes spawn and exit locations in the same maze family. Stage 3 introduces one wall shift every 20 seconds. Stage 4 increases the shift frequency to 8 seconds, raises shift intensity from one to three wall changes, and changes the training maze seed from 42 to 84. This is more precise than describing the legacy open-arena plot as the official curriculum.

All four stages use the same `reward_v5_active_agents` definition. A partial 590-episode Stage 1 run was invalidated when an audit found that earlier manifests switched reward definitions between static and dynamic stages. Such a switch would confound curriculum difficulty with reward design. Its first V5 retry was also invalidated because old and new append-only logs shared a run directory. Under simultaneous training, equal limits then froze the faster-accumulating three-agent Sentinel trainer before Runner finished. A 3:2-budget retry avoided that timing error but was also excluded: Sentinel wins fell from 92% in its first 100 episodes to 8% in its last 100 episodes, evidence of latest-policy cycling rather than stable improvement. Opponent-snapshot self-play corrected the simultaneous-update design, but a source-level audit showed that retaining 3:2 limits would instead create a final 500k Sentinel-only tail under the ghost controller's equal-step turns. That partial run was also invalidated. The final configuration combines snapshot self-play with equal per-policy sample limits. The launcher refuses official training unless reward, clean-retry, role-budget, and self-play audits pass.

TABLE VII
IMPLEMENTED FOUR-STAGE CURRICULUM. STEP BUDGETS ARE SENTINEL/RUNNER TRAINER STEPS.

Stage	Topology	Steps	Transfer
1. Static fixed	Static, seed 42	1.5M/1.5M	None
2. Static random	Static, seed 42	1.5M/1.5M	Stage 1
3. Dynamic low	20 s, intensity 1	1.5M/1.5M	Stage 2
4. Dynamic high	8 s, intensity 3	1.5M/1.5M	Stage 3
Evaluation	Seen + 5 held out	None	Stage 4

ML-Agents counts one wrapped-trainer step for each acting-agent transition. Its ghost controller changes the learning team after an equal number of wrapped-trainer steps and is designed to give every team the same trainer-step count. We therefore use 1.5M transition samples for each role per stage, so both optimizers receive equal sample budgets and finish self-play together. With role-level parameter sharing, these samples correspond to 0.5M Sentinel team decision cycles and 0.75M Runner team decision cycles because three Sentinels act per cycle but only two Runners do. We disclose both units rather than calling them equivalent. The matched direct-dynamic control uses 6M samples per role, exactly the aggregate four-stage curriculum budget.

C. Evaluation Protocol

We evaluate trained policies in inference mode using fixed ONNX checkpoints. The strengthened publication protocol uses five policy seeds (42, 101, 202, 606, 707), one seen dynamic-maze split, and at least five held-out unseen layout seeds (101, 202, 303, 404, 505). Each policy-layout cell targets 100 completed episodes. A 30-minute wall-clock cap is a failure timeout rather than the sampling unit. All evaluation runs use deterministic action selection with seeded spawn/episode randomization and must produce run-scoped raw logs: `episode_log.csv`, `agent_step_log.csv`, `reward_audit.csv`, and `replay_events.csv`. A run is excluded from publication tables if the episode target, any required log, metadata file, or KPI summary is missing.

The seen and held-out rule files are matched on agent speed, capture radius, timeout, wall-shift interval and intensity, sensing, zero action assistance, and all enabled randomization switches. Only the split identifier, topology seed, and placement/event seed differ. A machine-readable con-

trol audit compares the nested YAML fields before launch and rejects a matrix with any unregistered difference. The action-assist-on and dynamic-wall-off matrices use a parallel single-intervention audit: the former may change only the two assist coefficients, and the latter may change only `dynamic_walls.enabled`.

The evaluation protocol reports outcome metrics (Sentinel win rate, Runner win rate, escape rate, survival time, and capture times), coordination metrics (pincer episode rate, trap frequency, corridor-block events, exit-denial events, Sentinel spread, and Runner separation), path metrics (path efficiency and route-change proxies after wall shifts), and ablation deltas. Primary results use zero action assistance. The paired assist-on matrix quantifies how much the optional hybrid controller changes those results.

D. Metric Definitions

For event type m, let $C_{k,m}$ be its count in episode k. We distinguish an episode prevalence from an event intensity:

$$\widehat{P}_m = \frac{1}{n}\sum_{k=1}^{n} \mathbb{1}[C_{k,m} > 0], \qquad \overline{C}_m = \frac{1}{n}\sum_{k=1}^{n} C_{k,m}. \tag{11}$$

Calling both quantities a "rate" would be ambiguous, so exported summaries use names such as `pincer_episode_rate` and `pincer_events_per_episode`. Trap success is conditional:

$$\widehat{P}_{\text{trap}\to\text{win}} = \frac{\sum_k \mathbb{1}[C_{k,\text{trap}} > 0 \wedge y_k = S]}{\sum_k \mathbb{1}[C_{k,\text{trap}} > 0]}. \tag{12}$$

At step t, Sentinel spread is the mean pairwise distance among active Sentinels,

$$D_S(t) = \binom{|\mathcal{I}_S^t|}{2}^{-1} \sum_{i<j} \|x_i(t) - x_j(t)\|_2, \tag{13}$$

with an analogous Runner separation $D_R(t)$. Each spread is averaged over snapshots within an episode and then equally across episodes. Path length is $L_i = \sum_t \|x_i(t) - x_i(t-1)\|_2$, with positions keyed by episode and agent so resets are not counted as motion. Capture times come from replay-event `time_seconds`. Runner survival ends at that Runner's capture or at episode termination and is averaged over runner-episodes. For a wall shift at τ and one-second window w, route response is

$$\begin{aligned} \theta_{j,\tau} &= \cos^{-1}\!\left(\frac{u^- \cdot u^+}{\|u^-\|\|u^+\|}\right), \\ u^- &= x_j(\tau) - x_j(\tau - w), \\ u^+ &= x_j(\tau + w) - x_j(\tau). \end{aligned} \tag{14}$$

We report mean absolute deflection and the fraction with $\theta_{j,\tau} \geq 45^\circ$. Windows with less than 0.05 m displacement are excluded as directionally undefined.

E. Statistical Analysis Plan

The independent training seed is the unit of replication for policy-level claims. Episodes from the same checkpoint are not treated as independent policy replicates. We report each seed separately, then the mean and standard deviation across trained policies. Binary outcomes receive Wilson intervals within a policy/layout cell. For aggregate comparisons, a hierarchical bootstrap resamples policy seeds, then layouts, then episodes to preserve the nested design. Training uses the fixed `max_steps` budgets in Table VII. The YAML episode-count and win-rate thresholds are audited post hoc and do not terminate ML-Agents training.

Training curves are exported directly from each audited ML-Agents TensorBoard event file. Curves include cumulative reward, episode length, self-play ELO, entropy, policy loss, and value loss. They are aligned by role, curriculum stage, and normalized within-stage trainer progress. No run is admitted unless its completion audit passes. At every matched scalar step, the plot reports per-seed traces, the arithmetic mean, and a two-sided 95% Student-t interval across independent policy seeds. With five seeds the interval remains descriptive and potentially wide. Smoothing is limited to the summary aggregation already performed by ML-Agents.

The full condition contains $5 \times 6 \times 100 = 3{,}000$ completed evaluation episodes. Applying the same budget to five ablations yields a planned minimum of 18,000 completed episodes after all retrained checkpoints are available. This count does not create 18,000 independent policy replicates: inference about training variability remains limited to five policy seeds. The larger within-cell count instead stabilizes per-topology outcome and timing estimates.

For metric observation $z_{s,l,k}$ from policy seed s, layout l, and episode k, the layout-balanced estimator is

$$\widehat{M} = \frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}} \frac{1}{|\mathcal{L}|}\sum_{l\in\mathcal{L}} \left(\frac{1}{n_{s,l}}\sum_{k=1}^{n_{s,l}} z_{s,l,k}\right). \tag{15}$$

This prevents fast-terminating layouts from receiving more weight merely because they produce more episodes. The fixed-count protocol makes episode-weighted and layout-balanced estimates closer, but both are exported. Ablation effects are paired by training seed and evaluation layout:

$$\Delta_{a,s,l} = M_{\text{full},s,l} - M_{a,s,l}, \tag{16}$$

where a denotes an ablated component, s a policy seed, and l a layout. We report the mean paired effect, a 95% bootstrap interval, and the raw per-seed values. With only five policy seeds, interval width and effect consistency matter more than a binary significance threshold. For descriptive scale we also report the paired standardized effect

$$d_z = \overline{\Delta}_a / s(\Delta_a), \tag{17}$$

along with the fraction of matched seed-topology cells having the modal effect sign. We retain the raw metric units because d_z is unstable with a small policy-seed sample or when paired variance is near zero. The corresponding two-sided paired-t power function is

$$\pi(d, n) = \Pr\left(\left|T_{n-1,\, d\sqrt{n}}\right| > t_{1-\alpha/2,n-1}\right), \tag{18}$$

where $T_{\nu,\lambda}$ is noncentral t. At $\alpha = 0.05$, $n = 5$, and 80% power, numerical inversion gives $|d_z| \approx 1.68$. Thus even five seeds detect only large effects: more episodes improve within-cell precision but not policy-level power, and a nonsignificant seed test is not evidence of no effect. The exported power audit also reports thresholds of 3.26 for three seeds and 1.00 for ten.

For outcome metric M, the topology generalization gap for policy seed s is

$$G_s = M_{s,\text{seen}} - \frac{1}{5} \sum_{l \in \mathcal{L}_{\text{held}}} M_{s,l}. \tag{19}$$

For Sentinel win rate, $G_s > 0$ means Sentinel success decreases on held-out topologies. For Runner escape rate, $G_s > 0$ means Runner escape success decreases. We state the role perspective for every gap rather than assigning one global "favorable" direction. We report the five held-out values alongside G_s so that a favorable mean cannot hide a topology-specific failure. The statistics pipeline bootstraps each G_s by resampling the seen episodes and then held-out layouts and their episodes. It also exports a balanced two-way decomposition of policy-seed variation, layout variation, interaction, and finite-cell error from the cell means. No test treats the five layouts as five independently trained policies.

For held-out cell mean $\bar{z}_{s,l}$, the descriptive random-effects decomposition is

$$\bar{z}_{s,l} = \mu + a_s + b_l + u_{s,l}, \qquad \text{Var}(\bar{z}) = \sigma_a^2 + \sigma_b^2 + \sigma_u^2, \tag{20}$$

where a_s is policy-seed variation, b_l is layout variation, and $u_{s,l}$ combines seed and layout interaction with finite-cell estimation error. Method-of-moments components are truncated at zero and reported as variance fractions. With only five policy seeds they are descriptive rather than precise population estimates.

Memory, tactical-reward, and curriculum ablations alter the learning problem and therefore require paired retraining for every policy seed. Dynamic-wall and action-assist ablations are deployment-time interventions on the same checkpoints. Pairing the policy and episode/topology seeds isolates the immediate contribution of those mechanisms. We do not mix these two ablation types in one effect estimate.

The machine-readable suite pre-registers signs under the convention *canonical full minus condition*: shorter reacquisition with memory, higher tactical-event prevalence with tactical shaping, a smaller absolute transfer gap after curriculum training, higher Sentinel win rate with assist on, and stronger route deflection with dynamic walls. Metrics outside these hypotheses are explicitly exploratory. Disagreement with a registered sign is reported as a negative result rather than relabeled post hoc.

The memory intervention is a system-level ablation rather than a neural-input-only mask. It writes zeros into the fixed six-dimensional memory observation, stops last-known-target updates, and prevents the pursuit-assist controller from falling back to a remembered target when no opponent is visible. This preserves observation dimensionality and network compatibility, but any measured effect must be attributed to the complete explicit-memory subsystem.

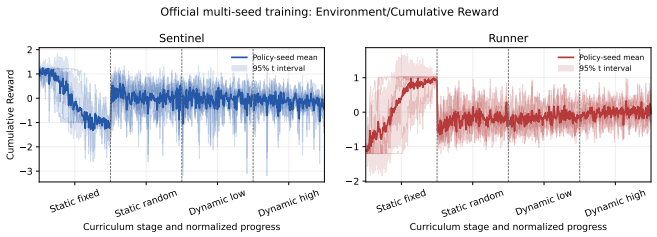

Fig. 4. Audited five-seed official curriculum reward curves exported from TensorBoard scalars. The official gate uses fixed trainer step budgets. The plotted curves are diagnostic summaries of completed training.

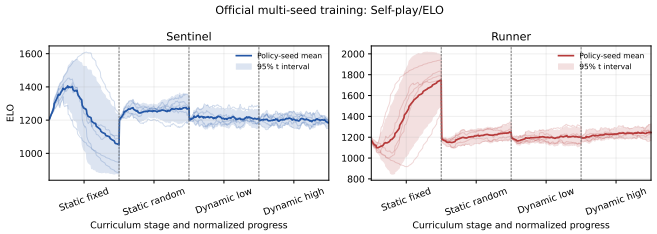

Fig. 5. Audited five-seed self-play ELO traces. Stable or increasing ELO does not imply Nash convergence. It is used as a training diagnostic.

VI. Current Official Evidence at Pause

Pause status. The strengthened campaign was intentionally paused on 2026-07-18 after completing the official curriculum baseline, the canonical assist-off evaluation, and two deployment-time paired ablations. The strict campaign tracker reported 47/65 training runs and 90/180 target-complete evaluation cells. The publication-readiness audit passed 14/16 gates. The two remaining blockers were official evaluation completeness for unfinished ablation families and paired ablation completeness: only two of five planned paired analyses were complete. Consequently, this section reports the completed evidence honestly and does not claim completion of the full journal experiment matrix.

The official five-seed curriculum baseline reached 20/20 audited training runs. Figure 4 summarizes the exported TensorBoard reward traces for the five independent policy seeds. The curve export contains 28,320 scalar rows and is generated from `results/official_summary/training_curves/`. It is an audited learning-curve artifact, not a stopping criterion: all official stages use fixed trainer step budgets.

Figure 5 reports the corresponding self-play ELO traces. ELO is useful for detecting cycling and relative opponent difficulty, but it is not treated as a game-theoretic equilibrium certificate.

A. Canonical Assist-Off Evaluation

The canonical evaluation uses the five trained curriculum policy seeds $(42, 101, 202, 606, 707)$, one seen topology, five held-out topologies, and a target of 100 completed episodes per policy-topology cell. This yields 30 completed cells and 3,002 episodes because two cells slightly exceeded the target before shutdown. Every reported row comes from the canonical eval-

TABLE VIII
PRE-REGISTERED ABLATION QUESTIONS AND ACCEPTANCE EVIDENCE. EACH CONDITION USES POLICY SEEDS 42, 101, 202, 606, AND 707 AND THE SAME HELD-OUT LAYOUTS.

Ablation	Controlled change	Primary metrics	Interpretation
Memory off	Retrain with the last-known-position subsystem disabled	Win/escape rate, capture time, reacquisition delay	Tests whether explicit system memory helps under occlusion
Tactical reward off	Retrain with trap/pincer/corridor/exit-denial rewards zeroed	Outcome rates and event prevalence/intensity	Separates coordination behavior from reward accounting
Dynamic walls off	Evaluate the same policies with matched topology/placement seeds	Survival, path length, route-change proxy	Measures the cost of within-episode topology changes
Direct dynamic training	Retrain without checkpoint transfer and match total environment steps	Learning curve, final outcome rates, seed variance	Tests the curriculum hypothesis
Action assist on	Evaluate the same policies with Sentinel pursuit blend restored to 0.15	All outcome and coordination metrics	Quantifies non-policy control contribution relative to the canonical assist-off evaluation

TABLE IX
CANONICAL ASSIST-OFF EVALUATION OVER FIVE POLICY SEEDS. VALUES ARE MEANS OVER POLICY-TOPOLOGY CELLS. THE SECOND NUMBER IS THE ACROSS-CELL STANDARD DEVIATION.

Metric	Seen	Five held-out topologies
Cells / episodes	5 / 501	25 / 2,501
Sentinel win rate	41.5% $\pm$ 5.6	42.2% $\pm$ 8.2
Runner win rate	58.5% $\pm$ 5.6	57.8% $\pm$ 8.2
Escape rate	52.9% $\pm$ 8.3	53.5% $\pm$ 9.6
Full capture time	23.70 $\pm$ 5.09 s	22.79 $\pm$ 4.04 s
Pincer episode rate	22.6% $\pm$ 4.5	20.3% $\pm$ 4.5
Corridor block rate	4.0% $\pm$ 1.8	2.5% $\pm$ 1.8
Exit denial rate	42.7% $\pm$ 3.3	39.2% $\pm$ 4.7
Trap success rate	49.4% $\pm$ 4.7	50.4% $\pm$ 8.2

uation aggregate and passed the strict publication-evaluation audit.

Table IX changes the interpretation of the earlier legacy story. Under canonical assist-off evaluation, the task is not Sentinel-dominated. Runner wins are more common than Sentinel wins on both the seen and held-out splits, and escapes account for roughly half of all episodes. This is evidence of a playable asymmetric benchmark with meaningful evasion, not evidence that the pursuer policy has solved the task. The held-out split is not materially worse than the seen split at this aggregate level, but the policy-seed variation is substantial: mean Sentinel win rate across the six layouts ranges from 31.6% for seed 101 to 50.8% for seed 606. This is why the paper reports per-seed results and avoids a robust-generalization claim.

B. Lightweight Evaluation-Only Baselines

To answer the reviewer request for a state-of-practice comparison without claiming an unearned retrained SOTA baseline, we added two evaluation-only controllers to the Unity runtime. The baseline selector is controlled by `LABYRINTH_BASELINE_POLICY` or by the runtime override file written by `scripts/evaluate_policy.py`. In `random` mode, both teams receive deterministic pseudo-random continuous actions keyed by episode, step, agent identifier, and seed. In `heuristic` mode, Sentinels greedily steer toward the nearest active Runner, while Runners blend steering away from the nearest Sentinel with steering toward the nearest exit. Both use local wall avoidance. The ML-Agents checkpoint path is still supplied only to maintain the standard inference handshake, but the Unity action override ignores the neural policy outputs.

Table X reports a 12-cell low-cost comparison using one evaluation seed and 25 target episodes per split. These rows are diagnostic, not a replacement for the five-seed canonical evaluation. They show that the learned PPO policy is substantially stronger than random pursuit on held-out capture time and coordination prevalence, but a hand-coded geometric controller still captures faster and wins more often. The defensible conclusion is therefore modest: the benchmark is not solved by the learned policy, and a future journal submission should include at least one matched-compute learned MARL baseline in addition to these rule-based controls.

C. Completed Paired Ablations

Two paired deployment-time interventions were complete at pause: action-assist on/off and dynamic-wall on/off. The convention is canonical full condition minus ablated condition. Thus, for *action assist on*, a negative Sentinel win delta means the canonical assist-off policy won less often than the assist-on hybrid controller. For *dynamic wall off*, a positive tactical event delta means the dynamic-wall condition produced more of that event than the static-wall intervention.

The completed action-assist intervention does not show a reliable win-rate benefit under the paired held-out analysis: the bootstrap interval spans zero. It does, however, change some spatial behavior. Canonical assist-off runs show lower Sentinel spread under the full-minus-assist-on convention. This supports disclosing assist settings but does not justify claiming that geometric assistance is necessary for success.

The dynamic-wall intervention has clearer tactical effects. With dynamic walls enabled, pincer episodes, exit denial, and trap episodes are higher on held-out topologies, and stall-step fraction is lower. The Sentinel win-rate delta still spans zero, so the defensible claim is behavioral: dynamic topology changes the coordination-event profile more clearly than it changes terminal win rate in the completed sample.

TABLE X
LOW-COST EVALUATION-ONLY BASELINES. LEARNED PPO USES THE CANONICAL FIVE-POLICY-SEED ASSIST-OFF EVALUATION FROM TABLE IX. RANDOM AND HEURISTIC ROWS USE ONE EVALUATION SEED, SIX SPLITS, AND 25 TARGET EPISODES PER SPLIT. HELD-OUT VALUES ARE MEAN ± STANDARD DEVIATION OVER FIVE LAYOUT CELLS.

Controller	Evidence	S win seen	S win held out	Escape seen	Escape held out	Full cap. held out	Pincer held out
Learned PPO	5 policy seeds, 30 cells, 3,002 episodes	41.5±5.6	42.2±8.2	52.9±8.3	53.5±9.6	22.79±4.04 s	20.3±4.5
Random actions	1 eval seed, 6 cells, 150 episodes	32.0	27.2±4.4	48.0	44.8±4.4	103.03±13.23 s	8.8±4.4
Geometric heuristic	1 eval seed, 6 cells, 152 episodes	73.1	68.3±9.7	26.9	30.1±9.5	8.74±0.59 s	26.3±13.0

TABLE XI
COMPLETED PAIRED ABLATIONS ON HELD-OUT TOPOLOGIES AT PAUSE. DELTAS USE THE FULL-MINUS-CONDITION CONVENTION AND ARE PAIRED BY POLICY SEED AND TOPOLOGY.

Ablation / metric	Delta	d_z	Boot. low	Boot. high
Assist on: Sentinel win	-1.26 pp	-0.17	-6.88 pp	4.44 pp
Assist on: escape	1.82 pp	0.25	-3.64 pp	7.38 pp
Assist on: Sentinel spread	-0.153 m	-0.48	-0.290	-0.011
Assist on: pincer rate	0.91 pp	0.16	-2.09 pp	4.50 pp
Wall off: Sentinel win	2.06 pp	0.29	-2.74 pp	7.22 pp
Wall off: pincer rate	2.75 pp	0.63	1.08 pp	4.52 pp
Wall off: exit denial	2.34 pp	0.45	0.20 pp	4.58 pp
Wall off: stall fraction	-6.46 pp	-1.21	-8.77 pp	-3.77 pp
Wall off: trap episode rate	3.16 pp	0.76	1.08 pp	5.40 pp

D. Interpretability from Existing Artifacts

The present manuscript does not report SHAP values or gradient saliency maps. Instead, it provides three layers of model interpretation that can be generated from the existing evidence pack. First, `reward_audit.csv` identifies which terminal, shaping, penalty, and tactical events contributed reward at each episode stage. This makes it possible to separate a real outcome change from a shaped-return increase. Second, `replay_events.csv` exposes behavioral mechanisms such as pincer pressure, corridor blocks, exit denial, captures, escapes, and wall shifts. Third, the paired action-assist and dynamic-wall interventions act as system-level counterfactual probes: the same policy, topology seeds, and placement seeds are replayed while changing one mechanism.

Under this interpretation lens, the current evidence is mixed and useful. The assist-on probe does not show a reliable Sentinel win-rate gain, which weakens any claim that the learned policy merely depends on geometric pursuit assistance. The wall-off probe changes tactical-event prevalence more clearly than terminal outcomes, suggesting that dynamic topology affects how agents coordinate even when it does not yet produce a statistically clear win-rate shift. This style of explanation is coarser than feature attribution, but it is well matched to MARL: it interprets behavior at the event, team, and mechanism levels where reviewers can connect policy outputs to pursuit-evasion concepts. Future SHAP-style analysis should group inputs by semantic channels, including self state, wall rays, exit rays, opponent rays, teammate rays, memory, and time context, rather than treating each scalar feature independently.

E. Unfinished Evidence

Three registered ablation families were not complete at pause: memory-off, tactical-reward-off, and direct-dynamic training. Memory-off and tactical reward ablations are retraining ablations, so they cannot be inferred from the canonical checkpoints. Direct-dynamic is also a retraining condition and has the largest remaining step budget. These incomplete gates are why the present PDF should be used for advisor review or a workshop/preprint direction, not as a final journal submission package.

VII. LEGACY EVIDENCE AUDIT (EXCLUDED FROM OFFICIAL RESULTS)

Exclusion notice. Every table and figure in this section is excluded from the official empirical study. The training plots use a legacy three-phase curriculum and legacy reward definitions. The fixed-checkpoint diagnostic executes a checkpoint named `v5_dynamicmaze`, but its inference episodes load `reward_dynamicmaze_memory_v4.yaml`, retain Sentinel action assistance at $\rho_S = 0.15$, reuse one hard-coded unseen topology, and apply a compound rule shift. These artifacts validate execution and provenance tooling only. They do not answer RQ1 to RQ3 and must be replaced by the registered v5 matrix.

TABLE XII
EXCLUDED LEGACY V4-ERA THREE-PHASE SUMMARY. HISTORICAL MOTIVATION ONLY.

Metric	Open+Static	Dynamic	Unseen
Episodes	3,228	2,492	30
Sentinel win rate	100.0%	66.8%	50.0%
Runner win rate	0.0%	33.2%	50.0%
Sentinel mean reward	+3.545	+0.680	+0.257
Runner mean reward	−2.407	−0.993	−0.181
Wall-loop penalty	−0.009	−0.364	−0.017
Cluster penalty	−0.013	−0.518	−0.035

The preliminary Sentinel win rate follows a curriculum-driven trend: near-perfect capture in the open arena/static maze setting, reduced dominance in the dynamic maze, and a balanced outcome in a small unseen-layout evaluation. We treat this as motivation rather than proof: stronger claims require repeated evaluation across policy seeds and multiple held-out layouts with confidence intervals.

A. Validated Checkpoint Diagnostic

After correcting standalone scene, rule-config, and reward-config routing, we evaluated the fixed `v5_dynamicmaze` checkpoint pair across one seen layout and five held-out seed configurations. Despite the checkpoint name, episode provenance confirms that inference used the legacy v4 reward file. Table IV therefore does not describe this diagnostic. Three evaluation seeds were used, but these are not independent training seeds. Each matrix cell ran for 30 wall-clock seconds in deterministic inference mode. The audit accepted all 18 cells, all 108 required raw/KPI artifacts, and every episode-level scene/rule/reward provenance check. A subsequent code audit found two design confounds. First, the five configurations reused one hard-coded unseen topology. Their seeds changed placements and stochastic events, not maze connectivity. Second, the legacy seen and unseen rule files changed agent speeds, capture radius, and wall timing together with the scene. They therefore measure transfer under a compound configuration shift, not a controlled topology effect.

TABLE XIII
EXCLUDED ASSIST-ON/V4 FIXED-CHECKPOINT DIAGNOSTIC. INTERVALS ARE 95% WILSON INTERVALS OVER COMPLETED EPISODES.

Metric	Seen	Five unseen configurations
Completed episodes	12	106
Sentinel wins	9 (75.0%)	73 (68.9%)
95% CI	[46.8, 91.1]%	[59.5, 76.9]%
Runner wins	3 (25.0%)	33 (31.1%)
Escape rate	0.0%	25.5%
Mean full capture time	13.33 s	21.04 s

The aggregate result remains Sentinel-favored, but the seed configuration matters: Sentinel win rate ranges from 33.3% under configuration 505 to 88.2% under configuration 202. Because topology was shared, this variation cannot be attributed to maze geometry. The seen-versus-unseen difference also cannot be assigned to topology because agent and wall dynamics changed simultaneously. The diagnostic instead motivates controlled topology generation, matched rule parameters, fixed-count evaluation, and independently trained policy seeds.

The canonical statistics pipeline also computes equal-configuration estimates. For the five unseen configurations, the Sentinel win rate is 68.9% when all 106 episodes are pooled and 68.3% when every configuration receives equal weight. Mean full-capture time changes from 21.04 s to 23.25 s under equal weighting. Policy-level hierarchical intervals are deliberately omitted for this diagnostic because all rows share one checkpoint pair. Nominal evaluation seeds do not create independent trained policies.

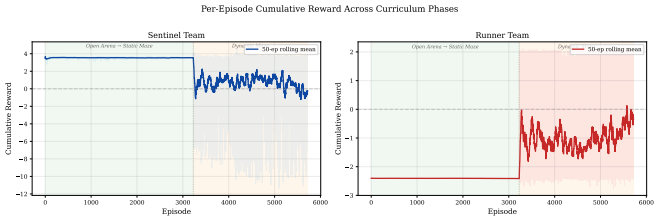

Fig. 6. Excluded legacy three-phase cumulative reward. Not comparable to the official v5 curriculum.

TABLE XIV
EXCLUDED ASSIST-ON/V4 RESULTS ON ONE SHARED UNSEEN TOPOLOGY. "FULL CAP." IS MEAN FULL-CAPTURE TIME OVER SENTINEL WINS.

Configuration	n	S wins	R wins	Esc.	Full cap.
Seen 42	12	75.0%	25.0%	0.0%	13.33 s
Unseen 101	31	80.6%	19.4%	19.4%	22.04 s
Unseen 202	17	88.2%	11.8%	0.0%	14.23 s
Unseen 303	12	75.0%	25.0%	25.0%	58.67 s
Unseen 404	28	64.3%	35.7%	32.1%	9.63 s
Unseen 505	18	33.3%	66.7%	50.0%	11.68 s

B. What the Diagnostic Does and Does Not Establish

The diagnostic is useful primarily as a test of executable transfer and measurement integrity. It establishes that the legacy ONNX pair runs in both requested scenes, that held-out rule files are actually loaded, and that complete episode, step, reward, replay, KPI, and metadata artifacts can be regenerated. It also identifies a concrete failure case: unseen seed 505 reverses the aggregate advantage and produces a 50% escape rate.

It is not a controlled generalization experiment. First, all cells reuse one policy pair. Second, a fixed 30-second wall-clock budget produces unequal sample counts from 12 to 31 episodes per configuration, so an episode-weighted aggregate gives greater influence to configurations with faster termination. Third, all unseen configurations reuse one topology, and deterministic inference makes nominal evaluation-seed repetitions partially redundant. Fourth, the legacy split comparison changes movement and wall parameters together with the scene. The official evaluation therefore uses a fixed completed-episode target per policy-topology cell, independent policy seeds, genuinely distinct seeded topologies, identical non-topology controls, and explicit spawn/episode randomization. Until those runs finish, Table XIV is a diagnostic table rather than the primary result of the paper.

C. Reward Curves

Figure 6 shows the excluded legacy per-episode cumulative reward for both teams across the earlier three-phase curriculum. The green-shaded region corresponds to the open arena and static maze phases, where the Sentinel reward is consistently high ($+3.545$). The orange-shaded region marks the transition to the dynamic maze, where the Sentinel mean reward drops sharply to $+0.680$. This historical trace is not evidence for the official four-stage curriculum.

Figure 7 decomposes the total reward into terminal, shaping, trap-aware, and penalty components. In the legacy run,

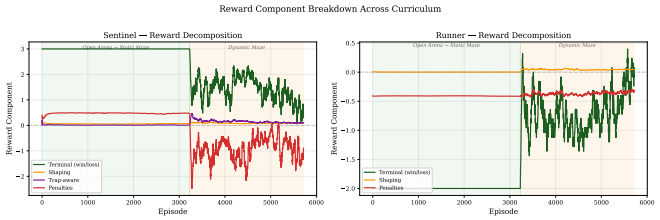

Fig. 7. Excluded legacy reward decomposition. Not evidence about the official v5 weights.

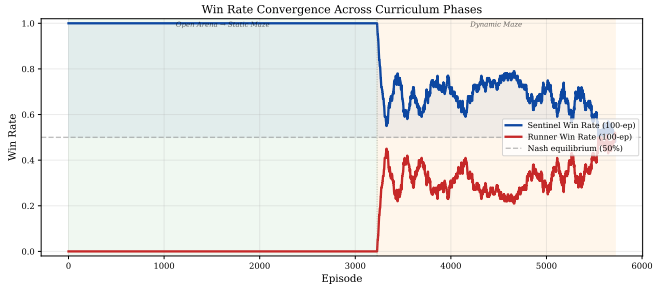

Fig. 8. Excluded legacy win-rate trace. Descriptive history rather than an official comparison.

terminal reward is the largest component, suggesting that shaping did not completely dominate win/loss learning. The publication pipeline verifies this claim by regenerating reward-breakdown tables from raw `reward_audit.csv` files for every run.

D. Win Rate Convergence

Figure 8 shows the excluded legacy 100-episode rolling win rate for both teams. In the static maze phase, the Sentinel win rate is near 100%. Upon entering the dynamic maze, the Runner win rate rises from near-zero toward a more balanced outcome. The trace motivates an evasion-viability hypothesis but cannot establish a dynamic-topology effect or equilibrium.

E. Penalty Reduction as Evidence of Learning

Figure 9 tracks penalty signals for both teams across training. The Sentinel cluster penalty, introduced to prevent free-rider collapse, shows that agents initially bunched together but gradually learned to spread out. The Runner threat-approach penalty decreases over training, indicating that Runners learned to maintain safe distances from Sentinels rather than passively waiting.

F. Shaping Signal Analysis

Figure 10 shows legacy distance-based shaping traces. Because the source runs predate the official v5 definition and the v4 diagnostic lacks the v5 idle and exit-approach terms, this plot cannot establish that v5 reduced passive evasion. It is retained only to document the hypothesis that motivated the registered reward ablation.

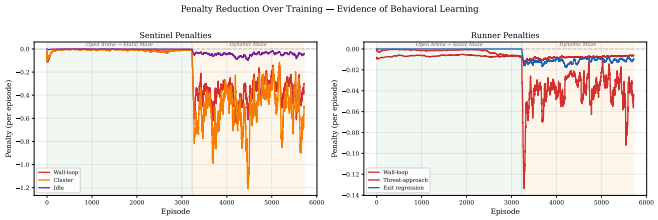

Fig. 9. Excluded legacy penalty traces. Not comparable to the official v5 reward definition.

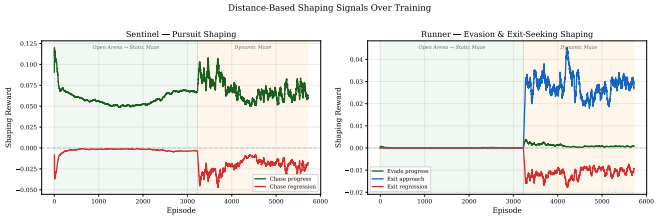

Fig. 10. Excluded legacy shaping traces. Mechanism attribution requires the official tactical-reward ablation.

G. Per-Phase Summary

Figure 11 presents the per-phase comparison as bar charts. The progression motivates a curriculum-learning hypothesis, but final zero-shot generalization claims require the expanded unseen-layout protocol.

H. Held-Out Configuration Diagnostic

The validated diagnostic confirms that the dynamic-maze checkpoint can execute in the held-out scene with seed-specific rule and reward configurations. Across 106 completed unseen-scene episodes, the aggregate Sentinel win rate is 68.9%, but the shared hard-coded topology means the spread from 33.3% to 88.2% is not a topology-generalization result. The upgraded generator now derives a connected maze from each held-out topology seed, separates topology seed from per-episode placement seed, and logs both the layout identifier and topology seed. The official experiment must rerun all five generated topologies with independently trained policies and fixed episode counts.

VIII. Discussion

The excluded audit motivates three hypotheses. It does not support completed findings.

Curriculum learning is a central hypothesis. The four-stage curriculum with transfer learning is designed to develop complex pursuit-evasion skills progressively. Its benefit is not established by the legacy traces and must be tested against direct dynamic-maze training with matched architecture and environment-step budget.

Custom reward shaping targets specific failure modes. The iterative development of the `v5_active_agents` reward configuration was driven by three observed failure modes:

- *Free-rider collapse:* In early training, one Sentinel learned to capture while others remained

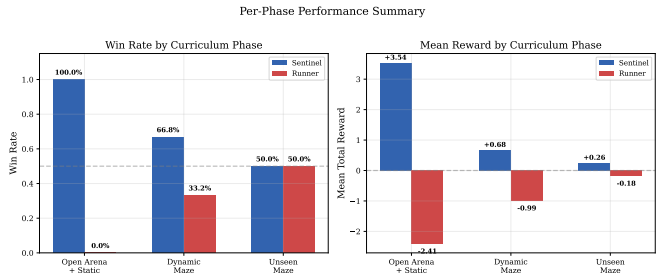

Fig. 11. Excluded legacy phase summary. It motivates but does not test the curriculum hypothesis.

idle to collect the shared team reward. The `idle_penalty` (-0.002/step) and increased `individual_capture_bonus` ($+0.25$) were added to reduce this behavior.

- *Cluster convergence:* All three Sentinels would chase the same Runner in single file, making evasion trivial. The `cluster_penalty` (-0.015) encourages spatial separation. Publication results must quantify this through Sentinel spread and pincer/corridor metrics.
- *Passive evasion:* Runners learned to survive by standing still, avoiding threat-approach penalties but never seeking exits. The `exit_approach_bonus` ($+0.005$) and `exit_approach_regression` (-0.002) were added to encourage goal-directed escape behaviour.

Generalization remains the key empirical test. The validated fixed-checkpoint diagnostic demonstrates executable transfer to a held-out scene under five seeded configurations, not five different topologies. The official protocol tests the upgraded procedural generator using independent policy seeds and five topology seeds, with topology identity recorded in every episode row.

Compared to MatrixWorld [9], which provides a grid-based pursuit-evasion benchmark, Labyrinth Breach operates in continuous 3D space with asymmetric teams, dynamic walls, and exit-based objectives. Compared to ViPER [13], which focuses on visibility-based pursuit, our environment adds team coordination pressure through the 3v2 asymmetry and tactical reward terms.

IX. Code, Data, and Reproducibility

The source repository contains the Unity project, ML-Agents trainer manifests, reward/rule/curriculum YAML files, Python launchers, audit scripts, unit tests, manuscript source, and generated summary artifacts. The public URL is given in the abstract footnote. The intended reproduction path is documented in `docs/reproducibility_guide.md`: build the Unity `6000.0.40f1` standalone, run the prelaunch validation gates, resume the official curriculum/evaluation/ablation launchers, export KPI summaries, rerun readiness audits, and compile the paper. The Python environment is specified by `environment.yml` and `requirements.txt`. The core versions are Unity ML-Agents package `com.unity.ml-agents@4.0.0`, Python `3.10.12`, `mlagents==1.1.0`, and `torch==2.2.1`.

The current PDF is generated from the journal LaTeX source under `paper/`. Official evaluation results are read from the publication-evaluation aggregate directory. Learning curves and paired ablations are read from the official-summary directory. The codebase includes unit tests under `tests/`. The last local validation for the publication pipeline previously passed the repository test suite, and the final manuscript build in this revision was rendered twice with `pdflatex`. The evidence-pack builder `build_publication_evidence_pack.py` creates a hashed archive with `artifact_manifest.json` and `SHA256SUMS`, but it is intentionally blocked for a formal submission pack until all readiness gates pass. This prevents an incomplete ablation matrix from being packaged as final journal evidence.

This revision also includes a low-effort random-action and geometric-heuristic baseline harness for evaluation-only diagnostics. Those rows are generated through the same fixed-checkpoint ML-Agents launch path and KPI summarizer, with Unity overriding policy actions at runtime. Their retained evidence level is KPI/metadata summaries plus aggregate CSV/JSON outputs, so they are reported as diagnostic controls rather than a formal matched-compute SOTA comparison. A full journal evidence pack should rerun the baseline cells with raw CSV log retention enabled or replace them with a learned MARL baseline under equalized compute.

X. Limitations and Future Work

Several limitations remain. First, the current study is simulation-only and does not include physical robot validation. Second, wall collision correctness under all dynamic shift configurations has not been exhaustively validated. Rare edge cases during wall transitions could allow agents to clip through geometry. Third, the tactical reward shaping terms add complexity to the reward landscape and could be exploited in ways not yet detected. Fourth, the current architecture uses last-known-position memory rather than recurrent neural memory, which may limit performance in highly dynamic environments. Fifth, the controller includes optional action-assist blending. Canonical results are assist-off and must be accompanied by the registered assist-on intervention.

XI. Conclusion

We presented Labyrinth Breach, a reproducible Unity ML-Agents benchmark for asymmetric multi-agent pursuit-evasion with dynamic topology, partial observation, exit-directed evasion, and tactical reward shaping. Legacy results motivate curriculum and reward hypotheses, but final claims require the registered multi-seed evaluation and ablations. The environment, configurations, logging pipeline, and checkpoints are public for reproduction and extension.

References

[1] AI641: AI for Robotics, "Projects," course handout, Project 2: Reinforcement Learning-based multi-agent pursuit-evasion.

[2] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," in *Advances in Neural Information Processing Systems*, 2017.

[3] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.

[4] C. Yu et al., "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games," in *Advances in Neural Information Processing Systems*, 2022.

[5] T. Verma, F. M. Zegers, and P. P. Khargonekar, "Multi-Agent Pursuer-Evader Learning using Situation Report," arXiv preprint arXiv:1910.07780, 2019.

[6] M. Zhou, Z. Liu, P. Sui, Y. Li, and Y. Y. Chung, "Learning Implicit Credit Assignment for Cooperative Multi-Agent Reinforcement Learning," in *Advances in Neural Information Processing Systems*, 2020.

[7] W. Chen, W. Li, X. Liu, S. Yang, and Y. Gao, "Learning Explicit Credit Assignment for Cooperative Multi-Agent Reinforcement Learning via Polarization Policy Gradient," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 10, pp. 11542–11550, 2023, doi: 10.1609/aaai.v37i10.26364.

[8] O. Gonultas and V. Isler, "Learning to Play Pursuit-Evasion with Dynamic and Sensor Constraints," arXiv preprint arXiv:2405.05372, 2024.

[9] L. Sun, Y.-C. Chang, C. Lyu, C.-T. Lin, and Y. Shi, "MatrixWorld: A Pursuit-Evasion Platform for Safe Multi-Agent Coordination and Autocurricula," arXiv preprint arXiv:2307.14854, 2023.

[10] M. Kouzeghar, Y. Song, M. Meghjani, and R. Bouffanais, "Multi-Target Pursuit by a Decentralized Heterogeneous UAV Swarm using Deep Multi-Agent Reinforcement Learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023.

[11] J. Chen, G. Li, C. Yu, X. Yang, B. Xu, H. Yang, and Y. Wang, "A Dual Curriculum Learning Framework for Multi-UAV Pursuit-Evasion in Diverse Environments," arXiv preprint arXiv:2312.12255, 2023.

[12] Y. Zheng, Y. Zhang, C. Zhao, H. Yang, T. Li, Q. Ouyang, and Y. Chen, "Faster Target Encirclement with Utilization of Obstacles via Multi-Agent Reinforcement Learning," in *Proc. Asian Conf. Mach. Learn.*, PMLR, vol. 222, pp. 1731–1746, 2024.

[13] Y. Wang, Y. Cao, J. Chiun, S. Koley, M. Pham, and G. A. Sartoretti, "ViPER: Visibility-based Pursuit-Evasion via Reinforcement Learning," in *Proceedings of the 8th Conference on Robot Learning*, PMLR, vol. 270, pp. 4188–4200, 2025.

[14] R. Lu, P. Zhang, R. Shi, Y. Zhu, D. Zhao, Y. Liu, D. Wang, and C. Alippi, "Equilibrium Policy Generalization: A Reinforcement Learning Framework for Cross-Graph Zero-Shot Generalization in Pursuit-Evasion Games," in *Advances in Neural Information Processing Systems*, 2025.

[15] R. Lu, R. Shi, Y. Zhu, and D. Zhao, "R2PS: Worst-Case Robust Real-Time Pursuit Strategies under Partial Observability," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.

[16] F. Yan, J. Jiang, K. Di, Y. Jiang, and Z. Hao, "Multiagent Pursuit-Evasion Problem with the Pursuers Moving at Uncertain Speeds," *Journal of Intelligent & Robotic Systems*, vol. 95, no. 1, pp. 119–135, 2019.

[17] J. Foerster, G. Farquhar, T. Afouras, V. Nardelli, and S. Whiteson, "Counterfactual Multi-Agent Policy Gradients," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, pp. 2974–2982, 2018, doi: 10.1609/aaai.v32i1.11794.

[18] R. Bhati, S. K. Gottipati, C. Mars, and M. E. Taylor, "Curriculum Learning for Cooperation in Multi-Agent Reinforcement Learning," in *Second Agent Learning in Open-Endedness Workshop at NeurIPS*, 2023.

[19] H. U. Sheikh and L. Bölöni, "Multi-Agent Reinforcement Learning for Problems with Combined Individual and Team Reward," arXiv preprint arXiv:2003.10598, 2020.

[20] Y. Zhang, M. Ding, J. Zhang, Q. Yang, G. Shi, M. Lu, and F. Jiang, "Multi-UAV Pursuit-Evasion Gaming Based on PSO-M3DDPG Schemes," *Complex & Intelligent Systems*, vol. 10, pp. 6867–6883, 2024, doi: 10.1007/s40747-024-01504-1.

[21] W. Du, T. Guo, J. Chen, B. Li, G. Zhu, and X. Cao, "Cooperative Pursuit of Unauthorized UAVs in Urban Airspace via Multi-Agent Reinforcement Learning," *Transportation Research Part C: Emerging Technologies*, vol. 128, Art. no. 103122, 2021, doi: 10.1016/j.trc.2021.103122.

[22] Z. Peng, G. Wu, B. Luo, and L. Wang, "Multi-UAV Cooperative Pursuit Strategy with Limited Visual Field in Urban Airspace: A Multi-Agent Reinforcement Learning Approach," *IEEE/CAA Journal of Automatica Sinica*, vol. 12, no. 7, pp. 1350–1367, 2025, doi: 10.1109/JAS.2024.124965.

[23] X. Qu, C. Li, Y. Jiang, F. Long, and R. Zhang, "Cooperative Pursuit of Unmanned Surface Vehicles Using Multi-Agent Reinforcement Learning," *Journal of Shanghai Jiaotong University (Science)*, vol. 31, pp. 187–194, 2026, doi: 10.1007/s12204-025-2816-6.

[24] S. Xu and Z. Dang, "Emergent Behaviors in Multiagent Pursuit Evasion Games within a Bounded 2D Grid World," *Scientific Reports*, vol. 15, Art. no. 29376, 2025, doi: 10.1038/s41598-025-15057-x.

[25] E. Cetin, C. Barrado, E. Salami, and E. Pastor, "Analyzing Deep Reinforcement Learning Model Decisions with Shapley Additive Explanations for Counter Drone Operations," *Applied Intelligence*, vol. 54, pp. 12095–12111, 2024, doi: 10.1007/s10489-024-05733-2.

[26] L. Sheng, X. Chen, Z. Pan, F. Cai, and H. Chen, "From Individual Decisions to Team Emergence: A Survey on Explainable Cooperative Multi-Agent Reinforcement Learning," *Artificial Intelligence Review*, 2026, doi: 10.1007/s10462-026-11598-3.

[27] P. K. Sharma, E. Zaroukian, D. E. Asher, and B. Howell, "Emergent Behaviors in Multi-Agent Target Acquisition," arXiv preprint arXiv:2212.07891, 2022.

[28] C. de Souza Jr., R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kulić, "Decentralized Multi-Agent Pursuit using Deep Reinforcement Learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4552–4559, 2021.

[29] N. A. Grupen, D. D. Lee, and B. Selman, "Multi-Agent Curricula and Emergent Implicit Signaling," in *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, pp. 560–568, 2021.

[30] R. Gorsane, O. Mahjoub, R. J. de Kock, R. Dubb, S. Singh, and A. Pretorius, "Towards a Standardised Performance Evaluation Protocol for Cooperative MARL," in *Advances in Neural Information Processing Systems*, vol. 35, pp. 5510–5521, 2022.

[31] A. Kapoor, B. Freed, J. Schneider, and H. Choset, "Assigning Credit with Partial Reward Decoupling in Multi-Agent Proximal Policy Optimization," in *Proceedings of the Reinforcement Learning Conference*, 2024.

[32] M. Akinmolayan, D. Josyula, and D. Casbeer, "Adaptive Interception in Dynamic Domains: Exploration of Hybrid Reinforcement Learning in Pursuit-Evasion Tasks," *Proceedings of the AAAI Spring Symposium Series*, vol. 8, no. 1, pp. 2–10, 2026, doi: 10.1609/aaaiss.v8i1.42510.

[33] X. Zhao and Y. Xie, "Multi-level Advantage Credit Assignment for Cooperative Multi-Agent Reinforcement Learning," in *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, PMLR, vol. 258, pp. 2971–2979, 2025.

[34] B. Zhao, Q. Guo, X. Wang, R. Hedjam, and G. Zhong, "MA2MB: Multi-Agent Mutual-Advising Model-Based Reinforcement Learning for Pursuit and Evasion Games," *Robotics and Autonomous Systems*, vol. 203, Art. no. 105530, 2026.